AI 1st Project

A L A P O

Final Delivery 1

Professor Ricardo Gomes

Gabriel Cardoso Mota - up202306287
Sofia Sousa Furtado - up202305878
Tiago Alexandre de Boaventura Nunes - up202304047

Approach – Heuristics and Evaluation Functions

To evaluate the Heuristics of a given board, we implemented one main function, evaluate_board. It calls the functions that will test the heuristics in individual fields and then calculate the total based on the given weights.

The material_eval_board returns a value that varies depending on the versatility of a piece, based on its shape (triangle – 1, square – 2, circle – 4). There's also a size multiplier of 5 for big pieces.

The position_with_table_eval_board calculates how good the position of a piece is. Seeing as the board is 6x6 and the objective is to reach the other side, this value will inscrease the further the piece is from the base rows, that correspond to 0, while the goal is 15.

Approach – Heuristics and Evaluation Functions

The safety_eval_board function uses the get_attack_map utils function to evaluate the safety of a piece. The attack maps return how many times each square is attacked by either pieces, using this, and dividing pieces into attackers and defenders. The function will also choose to preserve higher value pieces over lower ones.

Lastly, the control_eval_board funtion is responsible for ensuring that there's both pressure in the "home" rows of the opponent, but also a defensive presence in out part of the field, ensuring defense while attacking.

Operators and State Management

The Operators of our project insist upon the legal transitions of states, i.e., the legal moves given the pieces. As stated before, the operators are responsible for ensuring the squares can only move orthogonally, the triangles can only move diagonally and the circles are allowed to move in any of those directions. Each piece also has one of two sizes, big pieces may move how many spaces they can in the same direction, however, small pieces are limited to 1 space per turn.

In order to validate furhter, there’s an in bounds check to ensure things such as: a piece not being able to move past another piece, a piece not moving outsider the board, a piece not capturing pieces belonging to the same player, etc. All of these checks are performed by the move_piece function.

Search Algorithms

We implemented multiple adversarial search strategies, all based on Alpha-Beta pruning:

- Standard Alpha-Beta with Pruning
- Alpha-Beta with a transposition table
- Iterative Deepening with Alpha-Beta and transposition table

We found the most effective approach to be the one that combined:

- Iterative Deepening with Alpha-Beta
- Transposition Table
- History Heuristic
- Cutoff move per depth tracking
- Capture-based move ordering

This enhanced move ordering significantly reduced the number of nodes explored by prioritizing the most promising moves.

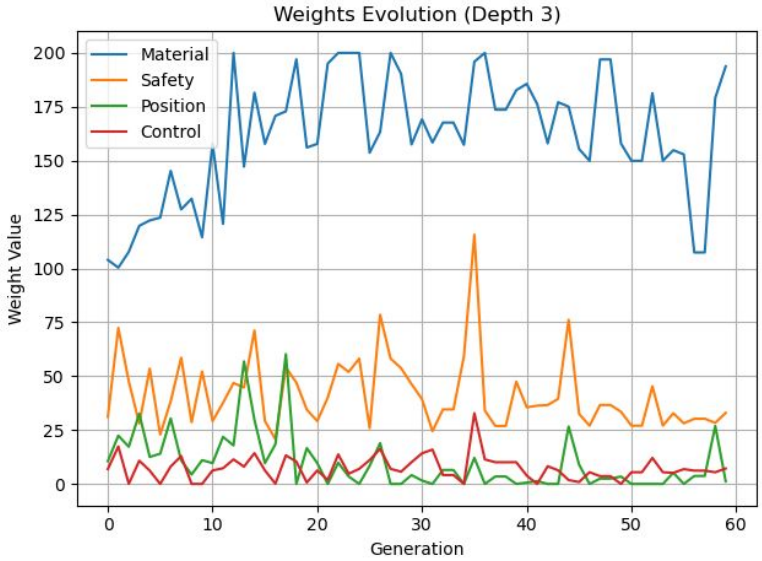
Experimental Results

To decide on the heuristics weights we ran simulated tournaments between AI's with different weight factors, keeping only the best performing and introducing some random mutations.

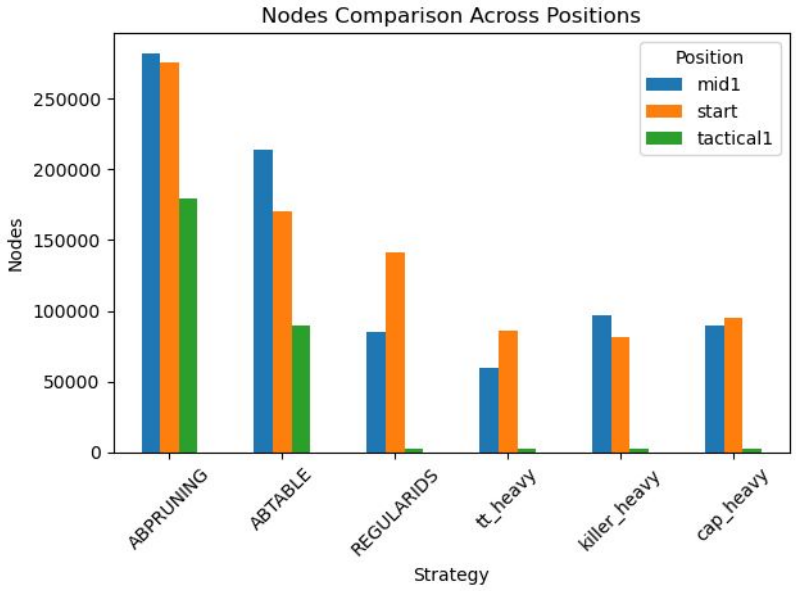
We noticed that:

- Material is heavely prioritized
- Safety shows moderate importance but fluctuates
- Position and Control have little impact on the evaluation

Since these tests were run on low depth we decided to settle on a slightly lower material weight, to give more importance to strategic play.



Experimental Results

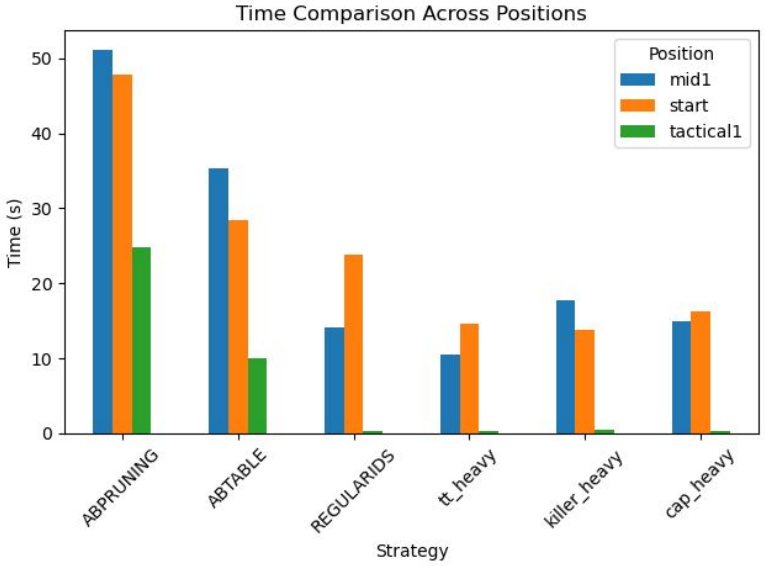


The graph shows the total number of nodes (or states) explored by each strategy for different positions.

As expected, as implement more sophisticated move ordering we achieve a lower node count.

Combining Interative Deepening Search and transposition tables also was a massive improvement as the best moves form previous depths guide deeper search

Experimental Results



Producing a Time Comparsion Graphic with the same variables as before gives us almost the exact same graph.

This makes sense and is in accordance with our expectations, since an algorithm that searches more nodes would naturally take a longer time to do so than an algorithm that is optimized to search fewer ones.

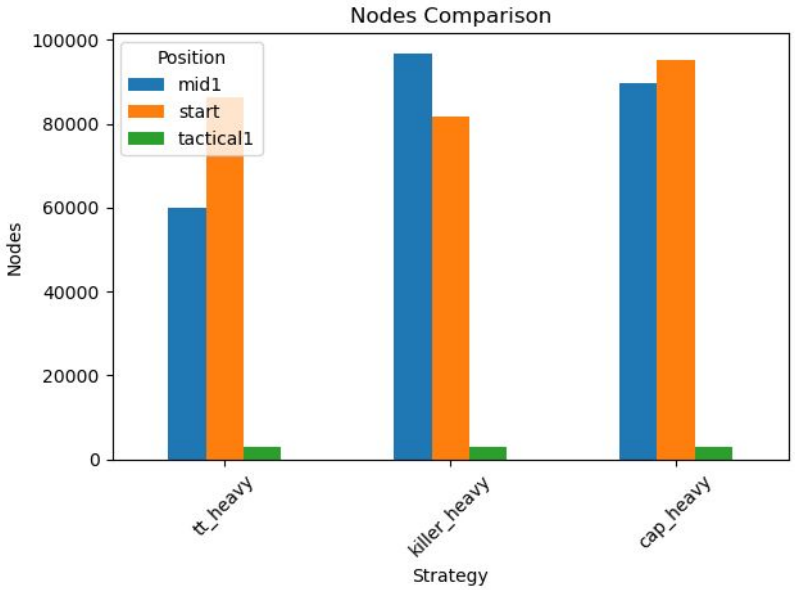
Experimental Results

To decide the order of moves to be explored we analyzed the number of nodes between 3 strategies with different ordering.

We concluded that:

- TT heavy is the best overall performing, shining in the mid-game
- Killer (Cutoff) heavy is the best on early stages
- Capture heavy is less reliable

Thus we decided a TT heavy Strategy was more appropriate for our AI, keeping the other weights as backup



References

All of the data for the graphs was obtained by running the program using scripts, test files with test functions and also test boards created specifically with the purpose of evaluating our intended data. The graphs themselves were created with basis on the information given, ploted in Google Sheets.